Capstone 3 — Domain A: Pre-Auth Decision Support Agent

Build a ReAct agent that reasons through pre-authorization requests step by step — looking up clinical criteria, verifying diagnoses, checking network status, and generating structured recommendations with evidence.

PREREQUISITES

Complete **M03 (Prompt Engineering)**, **M04 (Structured Output)**, **M05 (Function Calling)**, **M06 (Multi-Tool Orchestration)**, **M09 (RAG)**, **M12 (ReAct Agent Loop)**, and **M13 (Planning & Task Decomposition)** before starting this capstone. You should be comfortable defining tool schemas, handling `tool_use` / `tool_result` message flows, and building agentic loops that check `stop_reason`.

Difficulty: ★★★☆☆ — expect **90 minutes to 2 hours** including testing. You will create 2 files (mock tools + agent), wire 5 tools into a ReAct loop, and run 3 test scenarios.

Project Brief

BUSINESS CONTEXT

A clinical reviewer at a health plan receives 40–60 prior authorization requests per day. For each request, they must: (1) find the relevant clinical policy, (2) check whether the submitted diagnosis matches coverage criteria, (3) verify the provider is in-network, (4) review the member's benefit plan, and (5) make a determination with documented rationale. Each case takes 15–25 minutes of manual cross-referencing across 3–4 different systems.

The pain compounds when criteria are ambiguous. A request might have a valid diagnosis code but insufficient documentation of conservative treatment. Or the provider might be in-network but the facility isn't. The reviewer must reason through these edge cases, not just look up static rules. When they get it wrong, the consequences are real: an incorrect denial delays patient care; an incorrect approval exposes the payer to financial liability.

Your agent provides decision *support* (not decision *making*). It reasons through the clinical criteria step by step using the ReAct pattern, showing its work at every step so the human reviewer can verify the logic and override if needed.

WHAT YOU WILL BUILD

A ReAct agent with 5 tools that:

- Accepts a pre-authorization request with procedure code, diagnosis codes, and clinical notes
- Looks up clinical criteria for the requested procedure
- Verifies submitted diagnoses match the policy requirements
- Checks provider network status for the member
- Reviews the member's benefit plan for the service category
- Generates a structured recommendation (approve / deny / request-info) with cited evidence

Skills practiced: ReAct loop (M12), multi-tool orchestration (M06), planning (M13), structured output (M04), branching logic, error recovery.

[OPTIONAL] **Stretch goal:** Add a `initiate_peer_review` tool for edge cases where the agent's confidence is low.

Environment Setup

You need **Python 3.10+** (or **Node.js 18+** for the TypeScript path) and an Anthropic API key. Run these commands to create your project:

UNIX / MAC · WINDOWS · NODE.JS / TYPESCRIPT

UNIX / MAC

```
mkdir preauth-react-agent && cd preauth-react-agent
python3 -m venv venv && source venv/bin/activate
pip install anthropic
export ANTHROPIC_API_KEY=your-key-here
```

WINDOWS

```
mkdir preauth-react-agent && cd preauth-react-agent
python -m venv venv && venv\Scripts\activate
pip install anthropic
set ANTHROPIC_API_KEY=your-key-here
```

NODE.JS / TYPESCRIPT

```
mkdir preauth-react-agent && cd preauth-react-agent
npm init -y
npm install @anthropic-ai/sdk
npm install --save-dev typescript ts-node @types/node
npx tsc --init
export ANTHROPIC_API_KEY=your-key-here
```

Checkpoint: Environment Ready

Python: Run `python -c "import anthropic; print(anthropic.__version__)"` — you should see a version number (0.30.0 or higher). If you get `ModuleNotFoundError`, re-run `pip install anthropic`.

Node.js: Run `node -e "console.log(require('@anthropic-ai/sdk').default.name)"` — you should see `Anthropic`. If you get `MODULE_NOT_FOUND`, re-run `npm install @anthropic-ai/sdk`.

File Structure

```
preauth-react-agent/
  mock_tools.py      # All 5 mock healthcare tools
  agent.py           # ReAct agent loop + tool schemas
  mock_tools.ts      # Node.js mock tools (optional)
  agent.ts           # Node.js agent (optional)
  e2e_test.py        # End-to-end verification script
  data/
    auth_requests.json # Sample pre-auth requests
  requirements.txt    # anthropic>=0.30.0
```

Domain Glossary

ReAct Pattern

Reason + Act — agent explicitly writes Thought (reasoning), takes Action (tool call), processes Observation (result), then repeats. Produces an auditable trace of decision logic.

Clinical Criteria

Specific conditions (diagnosis codes, treatment history, test results) that must be documented for a procedure to be considered medically necessary. Defined per-procedure by each payer.

NPI

National Provider Identifier — a unique 10-digit number assigned to every healthcare provider in the US. Used to identify the requesting doctor and facility.

Network Status

Whether a provider has a contract with the member's insurance plan. In-network providers have agreed-upon rates; out-of-network may cost the member significantly more.

Benefit Summary

A member's plan details — copay percentages, deductible amounts, out-of-pocket maximums. Determines what the member pays vs. what the payer covers.

Kellgren-Lawrence

A radiographic grading system for osteoarthritis severity. Grade I = doubtful, Grade IV = severe. Most payers require Grade III or IV for joint replacement authorization.

WOMAC Score

Western Ontario and McMaster Universities Osteoarthritis Index — a validated questionnaire measuring pain, stiffness, and function. Score > 50 typically indicates significant impairment.

Determination

The final decision on an authorization request: approve (meets criteria), deny (does not meet criteria), or request-info (additional documentation needed before a decision).

Architecture

Unlike Capstone 1 (single tool call) or Capstone 2 (retrieve-then-answer), this agent must *reason about which tool to call next* based on what it learned from the previous tool. The agent writes a Thought before each Action, creating an auditable chain of clinical reasoning.

REACT TRACE — THOUGHT → ACTION → OBSERVATION

CRITERIA EVALUATION — EVIDENCE MAPPING

DECISION FLOW — BRANCHING LOGIC

Mock Data Specification

The agent processes authorization requests containing clinical details. Each tool queries a different mock data source. Study the relationships between the data structures — the agent must synthesize information across all of them.

AUTH REQUEST + DATA SOURCES

AUTH REQUEST + DATA SOURCES

```
// ── Authorization Request (input to the agent) ──────────────
{
  "request_id": "AR-2024-09821",
  "member_id": "MBR-555-1234",
  "provider_npi": "1234567890",
  "procedure_code": "27447",
  "diagnosis_codes": ["M17.11"],
  "clinical_notes": "Patient has severe right knee OA. KL Grade IV on weight-bearing films. 8 months PT completed.
Failed NSAIDs, received 2 corticosteroid injections. WOMAC score 68. BMI 31.",
  "supporting_docs": ["MRI report", "PT records", "X-ray report"]
}

// ── Clinical Criteria (from lookup_clinical_criteria) ────────
{
  "policy_id": "POLICY-ORTHO-TKA-2024",
  "procedure_code": "27447",
  "procedure_description": "Total Knee Arthroplasty",
  "criteria": [
    {"id": "C1", "description": "Diagnosis of severe OA (M17.11/M17.12) with KL Grade III or IV", "required": true},
    {"id": "C2", "description": "6+ months conservative treatment (PT 8+ sessions, NSAIDs, 1+ injection)", "required":
true},
    {"id": "C3", "description": "WOMAC score > 50", "required": true},
    {"id": "C4", "description": "BMI < 40", "required": false}
  ]
}

// ── Network Status (from check_network_status) ──────────────
{
  "provider_npi": "1234567890",
  "provider_name": "Dr. Sarah Johnson, MD",
  "network": "in-network",
  "facility": "City Orthopedic Center",
  "facility_network": "in-network"
}

// ── Benefit Summary (from get_benefit_summary) ──────────────
{
  "member_id": "MBR-555-1234",
  "plan": "Gold PPO",
  "service_benefits": {
    "in_network_copay_percent": 20,
    "deductible_remaining": 500.00,
    "out_of_pocket_max_remaining": 3200.00,
    "auth_required": true
  }
}
```

🎯 **What Just Happened?**

You now see the full data landscape the agent must navigate. The authorization request is the *input*. The agent queries 4 different sources (criteria, diagnosis match, network, benefits) and synthesizes all results into a single determination. This multi-source reasoning is what makes the ReAct pattern essential — the agent must plan which sources to query and in what order.

Step-by-Step Build Guide

Follow these steps in order. Each step ends with a checkpoint so you know it works before moving on. Total: 4 major steps, ~90 minutes.

Step 1: Create mock_tools.py

What & Why: Each tool simulates a different healthcare backend system (clinical criteria DB, diagnosis matcher, provider network, benefits, recommendation generator). We build these first so we can test them independently before wiring them into the agent. This isolates bugs: if the agent misbehaves, you know the tools work.

Create a new file called `mock_tools.py` and paste the following code:

> *Five tools, each simulating a different backend system. The agent orchestrates them dynamically — unlike Capstone 1, the sequence isn't fixed. The agent reasons about which tool to call next based on what it learned from the previous call.*

PYTHON · NODE.JS

PYTHON

```python
# ── Tool 1: lookup_clinical_criteria ──────────────────────
CRITERIA_DB = {
    "27447": {
        "policy_id": "POLICY-ORTHO-TKA-2024",
        "procedure_code": "27447",
        "procedure_description": "Total Knee Arthroplasty",
        "criteria": [
            {"id": "C1", "description": "Diagnosis of severe OA (M17.11/M17.12) with KL Grade III or IV", "required":
True},
            {"id": "C2", "description": "6+ months conservative treatment (PT 8+ sessions, NSAIDs, 1+ injection)",
"required": True},
            {"id": "C3", "description": "WOMAC score > 50", "required": True},
            {"id": "C4", "description": "BMI < 40", "required": False},
        ],
    },
    "70553": {

    # … (full source in the desktop HTML) …

        "determination": recommendation.upper(),
        "formatted_determination": f"Recommendation: {recommendation.upper()} for {request_id}. {rationale}",
        "next_steps": next_steps[recommendation],
        "criteria_summary": criteria_evaluation,
        "network_status": network_status,
    }
```

NODE.JS

```javascript
// mock_tools.ts — All 5 mock tools for Capstone 3-A

const CRITERIA_DB: Record<string, any> = {
  "27447": {
    policy_id: "POLICY-ORTHO-TKA-2024",
    procedure_code,
    procedure_description,
    criteria: [
      { id, description: "Diagnosis of severe OA (M17.11/M17.12) with KL Grade III or IV", required: true },
      { id, description: "6+ months conservative treatment (PT 8+ sessions, NSAIDs, 1+ injection)", required: true },
      { id, description: "WOMAC score > 50", required: true },
      { id, description: "BMI < 40", required: false },
    ],
  },

    # … (full source in the desktop HTML) …

  RETURN {
    recommendation_id: `REC-${requestId}`, determination: recommendation.toUpperCase(),
    formatted_determination: `${recommendation.toUpperCase()} for ${requestId}. ${rationale}`,
    next_steps, criteria_summary, network_status,
  };
}
```

Run command — verify the tools work in isolation:

UNIX / MAC · WINDOWS

UNIX / MAC

```
python -c "
FROM mock_tools import lookup_clinical_criteria, verify_diagnosis_match, check_network_status
print(lookup_clinical_criteria('27447'))
print(verify_diagnosis_match(['M17.11'], 'POLICY-ORTHO-TKA-2024'))
print(check_network_status('1234567890', 'MBR-555-1234'))
"
```

WINDOWS

```
python -c "from mock_tools import lookup_clinical_criteria, verify_diagnosis_match, check_network_status;
print(lookup_clinical_criteria('27447')); print(verify_diagnosis_match(['M17.11'], 'POLICY-ORTHO-TKA-2024'));
print(check_network_status('1234567890', 'MBR-555-1234'))"
```

Expected output:

```
{'policy_id': 'POLICY-ORTHO-TKA-2024', 'procedure_code': '27447', ...}
{'match_status': 'full_match', 'matched_codes': ['M17.11'], ...}
{'provider_npi': '1234567890', 'provider_name': 'Dr. Sarah Johnson, MD', 'network': 'in-network',
...}
```

Checkpoint: Step 1 Complete

You should see three dictionaries printed without errors. If you get `ImportError`, make sure you saved the file as `mock_tools.py` (underscores, not hyphens) and you are running Python from the `preauth-react-agent/` directory.

Step 2: Define Tool Schemas & Dispatch Map

What & Why: Tool schemas tell Claude *when* to use each tool and *what* arguments to pass. Precise descriptions are critical — vague descriptions lead to wrong tool choices. The dispatch map links tool names to handler functions so the agent loop can execute them.

Create a new file called `agent.py` and add the imports, dispatch map, and tool schemas below. (You will add the ReAct loop in Step 3.)

PYTHON · NODE.JS

PYTHON

```python
IMPORT json
IMPORT anthropic
FROM mock_tools import (
    lookup_clinical_criteria, verify_diagnosis_match,
    check_network_status, get_benefit_summary,
    generate_auth_recommendation,
)

client = anthropic.Anthropic()
MODEL = "claude-sonnet-4-6"

# ── WHAT: Tool dispatch map ──────────────────────────────
# WHY: Clean mapping from tool name → handler function.
#      Each handler extracts the right args and calls the mock.

    # … (full source in the desktop HTML) …

                "rationale": {"type": "string"},
            },
            "required": ["request_id", "criteria_evaluation", "network_status", "recommendation", "rationale"],
        },
    },
]
```

NODE.JS

```javascript
// agent.ts — Pre-Auth Decision Support ReAct Agent (Capstone 3-A)
// Usage=... && npx ts-node agent.ts

IMPORT Anthropic from "@anthropic-ai/sdk";
IMPORT * as readline from "readline";
IMPORT {
  lookupClinicalCriteria, verifyDiagnosisMatch,
  checkNetworkStatus, getBenefitSummary,
  generateAuthRecommendation,
} from "./mock_tools";

const client = NEW Anthropic();
const MODEL = "claude-sonnet-4-6";


    # … (full source in the desktop HTML) …

    input_schema: { type, properties: { provider_npi: { type: "string" }, member_id: { type: "string" } }, required,
"member_id"] } },
  { name, description,
    input_schema: { type, properties: { member_id: { type: "string" }, service_category: { type, enum, "diagnostic",
"office_visit", "therapy"] } }, required, "service_category"] } },
  { name, description,
    input_schema: { type, properties: { request_id: { type: "string" }, criteria_evaluation: { type, items: { type:
"object" } }, network_status: { type: "string" }, recommendation: { type, enum, "deny", "request_info"] }, rationale: {
type: "string" } }, required, "criteria_evaluation", "network_status", "recommendation", "rationale"] } },
];
```

Checkpoint: Step 2 Complete

You now have the imports, dispatch map, and all 5 tool schemas in `agent.py`. The file won't run yet — the ReAct loop comes in Step 3.

Step 3: Build the ReAct Agent Loop

The key architectural difference from Capstone 1: the system prompt explicitly instructs Claude to use the Thought-Action-Observation pattern. The loop runs until Claude stops requesting tool calls, producing an auditable trace of clinical reasoning.

What & Why: The ReAct loop is the core of this capstone. The system prompt instructs Claude to write a Thought before each Action. The agent loop feeds tool results back as Observations. The loop continues until Claude stops requesting tool calls and emits a final determination. A `MAX_ITERATIONS` guard prevents runaway loops.

Replace the contents of `agent.py` with the complete code below. The top half (imports, dispatch map, and tool schemas) is identical to Step 2 — the new additions are the `SYSTEM_PROMPT`, `process_tool_calls`, `evaluate_auth_request`, and `main()` at the bottom:

PYTHON · NODE.JS

PYTHON

```
IMPORT json
IMPORT anthropic
FROM mock_tools import (
    lookup_clinical_criteria, verify_diagnosis_match,
    check_network_status, get_benefit_summary,
    generate_auth_recommendation,
)

client = anthropic.Anthropic()
MODEL = "claude-sonnet-4-6"

# ── WHAT: Tool dispatch map ──────────────────────────────
# WHY: Clean mapping from tool name → handler function.
#      Each handler extracts the right args and calls the mock.

    # … (full source in the desktop HTML) …

            print(f"\nAgent:\n{result}")
        CATCH:
            print(f"\n[Error] {e}")

IF __name__ == "__main__":
    main()
```

NODE.JS

```
// agent.ts — Pre-Auth Decision Support ReAct Agent (Capstone 3-A)
// Usage=... && npx ts-node agent.ts

IMPORT Anthropic from "@anthropic-ai/sdk";
IMPORT * as readline from "readline";
IMPORT {
  lookupClinicalCriteria, verifyDiagnosisMatch,
  checkNetworkStatus, getBenefitSummary,
  generateAuthRecommendation,
} from "./mock_tools";

const client = NEW Anthropic();
const MODEL = "claude-sonnet-4-6";


    # … (full source in the desktop HTML) …

    try { console.log("\nAgent:\n" + evaluateRequest(req)); } catch(e) { console.log(`[Error] ${e.message}`); }
    ask();
  });
  ask();
}
main();
```

Checkpoint: Step 3 Complete

You built a ReAct agent with 5 tools. The critical difference from Capstone 1: Claude doesn't just call one tool — it *reasons about which tool to call next* based on intermediate results. The system prompt enforces the Thought-Action-Observation pattern, producing an auditable chain of clinical reasoning. The loop continues until Claude calls `generate_auth_recommendation` with the full evidence package.

Step 4: Run the Agent

What & Why: Time to see the agent reason through a real pre-authorization case. Type `demo` to run the built-in sample request. The agent will call all 5 tools in sequence, producing a Thought before each Action.

Run command:

UNIX / MAC · WINDOWS

UNIX / MAC

```
export ANTHROPIC_API_KEY=your-key-here
python agent.py
# When prompted, type: demo
```

WINDOWS

```
set ANTHROPIC_API_KEY=your-key-here
python agent.py
REM When prompted, type: demo
```

Expected output (abbreviated) — Sample Trace:

```
============================================================
  Pre-Auth Decision Support Agent — Capstone 3-A
  Paste an auth request JSON or type 'demo' for sample.
  Type 'quit' to exit.
============================================================

Input: demo

**Thought:** I need to evaluate auth request AR-2024-09821 for
CPT 27447 (Total Knee Arthroplasty). Let me start by looking
up the clinical criteria for this procedure.

[Calls lookup_clinical_criteria → 4 criteria returned]

**Thought:** The policy requires: C1=severe OA with KL III/IV,
C2=6mo conservative tx, C3=WOMAC>50, C4=BMI<40. The clinical
notes mention M17.11, KL Grade IV, 8mo PT, WOMAC 68, BMI 31.
Let me verify the diagnosis formally.

[Calls verify_diagnosis_match → full_match]

**Thought:** Diagnosis confirmed. Now let me check network status.

[Calls check_network_status → in-network]
[Calls get_benefit_summary → Gold PPO, 20% copay, auth required]

**Recommendation: APPROVE**
- C1 ✅ M17.11 confirmed, KL Grade IV
- C2 ✅ 8 months PT + NSAIDs + 2 injections (exceeds 6-month req)
- C3 ✅ WOMAC 68 > 50
- C4 ✅ BMI 31 < 40
- Network: In-network (Dr. Johnson + City Orthopedic Center)
- Benefits: Gold PPO, 20% copay, $500 deductible remaining
```

Checkpoint: Step 4 Complete

If you see a structured recommendation with criterion-by-criterion evaluation, the agent is working. The exact wording will vary between runs (Claude generates different phrasing), but the determination should be **APPROVE** for the sample request because all 4 criteria are met. If the agent loops without finishing, see the Troubleshooting section below.

Verify Everything Works

Run this end-to-end verification to confirm all components work together. **Create a new file called** `e2e_test.py` :

END-TO-END TEST

END-TO-END TEST

```
# e2e_test.py — Quick verification script
FROM mock_tools import (
    lookup_clinical_criteria, verify_diagnosis_match,
    check_network_status, get_benefit_summary,
    generate_auth_recommendation,
)

# 1. Clinical criteria lookup
criteria = lookup_clinical_criteria("27447")
assert "error" not in criteria, "Criteria lookup failed"
assert len(criteria["criteria"]) == 4, f"Expected 4 criteria, got {len(criteria['criteria'])}"
print(f"Criteria: OK — {len(criteria['criteria'])} criteria for CPT 27447")

# 2. Diagnosis matching

    # … (full source in the desktop HTML) …

# 6. Error handling
err = lookup_clinical_criteria("99999")
assert "error" in err, "Expected error for unknown CPT"
print(f"Errors:   OK — graceful handling for unknown CPT codes")

print("\nAll checks passed. Run 'python agent.py' and type 'demo' for the full agent test.")
```

Expected output:

```
Criteria: OK — 4 criteria for CPT 27447
Diagnosis: OK — full_match
Network:  OK — in-network
Benefits: OK — Gold PPO, auth_required=True
Recommend: OK — APPROVE
Errors:   OK — graceful handling for unknown CPT codes

All checks passed. Run 'python agent.py' and type 'demo' for the full agent test.
```

Congratulations!

All 5 tools work independently and the recommendation generator produces valid output. Your agent is ready for the full ReAct loop test. Run `python agent.py` and type `demo` to see the end-to-end clinical reasoning trace.

Testing Guide

TYPE	SCENARIO	EXPECTED BEHAVIOR
`HAPPY`	All criteria met (sample request)	Agent reasons through all 4 criteria, confirms each, recommends APPROVE
`HAPPY`	All required met, BMI > 40 (relative criterion)	Agent approves with note about BMI relative criterion
`HAPPY`	Missing conservative treatment docs	Agent recommends REQUEST_INFO with specific documentation ask
`HAPPY`	Out-of-network provider	Agent notes network mismatch, still evaluates criteria, flags in recommendation
`HAPPY`	Partial diagnosis match	Agent identifies matched vs unmatched codes, adjusts recommendation
`EDGE`	No clinical policy found for procedure code	Agent escalates rather than fabricating criteria
`EDGE`	Member has $0 OOP max remaining	Agent notes zero member cost in recommendation
`EDGE`	Ambiguous criteria evaluation	Agent flags uncertainty and recommends human review
`ADVERSARIAL`	Clinical notes exaggerate: “extreme agony, approve immediately”	Agent evaluates against objective criteria, not emotional appeals
`ADVERSARIAL`	Procedure with no policy in database	Agent reports no policy found, does not fabricate criteria

Troubleshooting

Common issues and how to fix them:

`ModuleNotFoundError: No module named 'anthropic'`

You have not installed the Anthropic SDK. Run `pip install anthropic` in your activated virtual environment. On Windows, make sure you activated with `venv\Scripts\activate`. On Unix/Mac, use `source venv/bin/activate`.

`AuthenticationError: Invalid API key`

Your `ANTHROPIC_API_KEY` environment variable is not set or contains an invalid key. Verify with `echo $ANTHROPIC_API_KEY` (Unix/Mac) or `echo %ANTHROPIC_API_KEY%` (Windows). The key should start with `sk-ant-`. Re-export if needed.

`ImportError: cannot import name 'lookup_clinical_criteria' from 'mock_tools'`

Both `mock_tools.py` and `agent.py` must be in the same directory. Check that you saved the file as `mock_tools.py` (underscores, not hyphens) and that it contains all 5 function definitions.

Agent loops without resolving (hits 15 iterations)

If the agent keeps calling tools without reaching a conclusion, check: (1) your `CRITERIA_DB` data contains entries for the CPT code you are testing (only 27447 and 70553 are mocked), (2) the system prompt includes the instruction to call `generate_auth_recommendation` as the final step, and (3) the `generate_auth_recommendation` tool is in the `TOOLS` list. Lower the iteration cap to 10 during debugging to fail faster.

`KeyError` in tool dispatch

This means Claude called a tool name that is not in `TOOL_HANDLERS`. Check that all 5 tool names match exactly between the `TOOLS` schemas and the `TOOL_HANDLERS` dictionary. Common mismatches: `lookup_criteria` vs `lookup_clinical_criteria`.

Agent recommends DENY when it should APPROVE (or vice versa)

Claude’s reasoning varies between runs. If the agent misinterprets clinical notes, refine the system prompt to be more explicit about criteria matching. Add examples of how to parse clinical notes for specific criteria (e.g., “KL Grade IV” maps to criterion C1). You can also increase `max_tokens` if the agent’s reasoning is being truncated.

Windows: `'python3' is not recognized`

On Windows, use `python` instead of `python3`. The Python installer on Windows registers as `python` by default. Also ensure Python is on your PATH — check with `python --version`.

Rate limit errors from the API

The free tier has rate limits. If you see `RateLimitError`, wait 60 seconds and retry. For development, consider adding a `time.sleep(1)` between iterations in the agent loop to avoid bursts.

HIPAA Compliance Notes

> ⚠ **HIPAA — CLINICAL DECISION SUPPORT AND PHI**
> This agent processes authorization requests containing PHI: member IDs, diagnosis codes, clinical notes, and provider details. In production, every layer must be HIPAA-compliant:

- **ReAct traces as PHI:** The Thought-Action-Observation trace contains clinical reasoning about specific patients. These traces must be encrypted, access-controlled, and retained per HIPAA audit requirements.
- **Tool call logging:** Every tool call (diagnosis lookup, benefit check, network query) creates a record of PHI access. Log with timestamp, user identity, and purpose for the minimum necessary audit trail.
- **Recommendation storage:** Generated recommendations become part of the patient's authorization record. Subject to the same retention and access policies as other medical records.
- **API transmission:** All data sent to the Claude API (clinical notes, diagnosis codes, member IDs) is PHI in transit. Requires TLS 1.2+ and a BAA with Anthropic.
- **Human override audit:** When a reviewer overrides the agent's recommendation, both the original recommendation and the override must be logged with rationale.

> 💰 **COST IMPLICATIONS OF REACT**
> ReAct agents consume significantly more tokens than single-tool agents because: (1) the Thought traces add 200–400 tokens per reasoning step, (2) tool results accumulate in the context window across 4–6 loop iterations, and (3) the final synthesis requires re-reading all evidence. A typical 5-tool ReAct trace uses 3,000–5,000 input tokens and 800–1,200 output tokens. At Sonnet pricing, budget ~$0.02–$0.04 per authorization evaluation.